

TRABAJO INTEGRADOR ABP - MÓDULO PROGRAMADOR

EVIDENCIA 5 - HITO 2: MODELO DE DATOS Y PROTOTIPO NAVEGABLE

Sistema de Gestión de Reparaciones de PC "TecnoService"

Carrera: Tecnicatura Superior en Desarrollo de Software (ISPC)

Equipo: Bit&Fix | Comisión: COM B1

Integrantes: KIEFFER, Ricardo; JERONIMO, Juan; BILEISIS, Joaquin; ESPINOSA, Thiago

Repositorio GitHub: <https://github.com/ricardokieffer/Bit-Fix-TecnoService>

Fecha de Entrega: Septiembre 2026

1. Presentación y Repositorio Git

El presente informe documenta las entregas del Hito 2 (Evidencia 5) del proyecto TecnoService. Reúne el modelo de datos relacional normalizado en 3FN, el script DDL con roles y datos de prueba, el diccionario de datos y el prototipo navegable en Python con Tkinter.

El repositorio oficial del grupo cuenta con commits de todos los integrantes, garantizando la autoría y distribución equitativa del trabajo en equipo.

2. Motor de Base de Datos

El proyecto utiliza PostgreSQL como motor relacional, conforme a los Requisitos Técnicos Obligatorios de la cátedra ("Motor relacional: PostgreSQL o MySQL"). El script `script_tecnoservice.sql` está escrito íntegramente en sintaxis de PostgreSQL (tipos SERIAL, bloques DO \$\$...\$\$ para la creación condicional de roles), y el módulo `conexion.py` está preparado con la librería `psycopg2` para conectarse a una instancia de PostgreSQL en el Hito 3.

3. Estructura Modular del Código Python (Tkinter)

Siguiendo los requisitos de la cátedra, el código se encuentra dividido modularmente en 4 archivos principales:

- `main.py`: Punto de entrada principal que ejecuta la aplicación y el bucle de eventos `tk.Tk()`.
- `interfaz.py`: Construye la ventana de escritorio con `ttk.Notebook` y las 5 pestañas navegables (Clientes, Ingresos, Órdenes, Técnicos, Repuestos) con formularios, botones y listas tabulares `ttk.Treeview`.
- `acceso_datos.py`: Capa de datos mock para interactuar de forma inmediata desde la interfaz en esta etapa del proyecto.
- `conexion.py`: Módulo preparado para conectar con PostgreSQL mediante `psycopg2`.

Estado de la integración: en este Hito 2 la aplicación funciona como un prototipo navegable con datos de prueba en memoria (`AccesoDatosMock`), aún sin conexión real a la base de datos, tal como establecen los lineamientos para esta instancia ("Aplicación Tkinter navegable con las ventanas principales construidas, aún sin conexión a la base"). La conexión real mediante `conexion.py` y las sentencias SQL parametrizadas se integrarán en el Hito 3.

4. Coherencia del Modelo de Datos

El modelo de datos separa claramente los datos del hardware (tabla equipo) de los datos de cada visita al taller (tabla reparacion). La falla reportada por el cliente y el estado del proceso se registran en `reparacion` y no en `equipo`, porque un mismo equipo puede ingresar más de una vez al taller con fallas distintas en cada oportunidad.

(cardinalidad equipo 1:N reparacion). Este criterio es el que sigue el diccionario de datos, el script DDL y el prototipo Tkinter, de forma consistente entre los tres.

Los dominios de valores fijos (tipo de equipo, especialidad del técnico y estado de la reparación) están unificados entre el script SQL (restricciones CHECK), el diccionario de datos y los combobox de la interfaz, para evitar que un dato cargado desde la aplicación viole una restricción de la base al integrarse en el Hito 3.

5. Conformación y Roles del Equipo Bit&Fix

Integrante	Rol Principal	Responsabilidad Hito 2
KIEFFER, Ricardo	Coordinador & Validaciones	Repositorio GitHub, estructura de main.py y validación de formularios Tkinter.
JERONIMO, Juan	Modelo de Datos & Acceso SQL	Script SQL DDL (script_tecnoservice.sql) en PostgreSQL, roles, reglas CHECK e INSERTs.
BILEISIS, Joaquin	Interfaz Gráfica (UI)	Prototipo navegable en interfaz.py con ttk.Notebook (5 pestañas) y Treeview.
ESPINOSA, Thiago	Documentación y Pruebas	Informe final Hito 2, diccionario de datos y pruebas de navegación del prototipo.

6. Devolución Docente y Ajustes Incorporados

A partir de la devolución sobre este Hito 2, se incorporaron los siguientes documentos y funcionalidades, previamente pendientes:

- Minuta de Requerimientos (Minuta_de_Requerimientos_TecnoService.docx): responde puntualmente qué pasa si un equipo reingresa por otra falla, qué sucede si falta stock de un repuesto, y cómo se registra la aceptación o el rechazo del presupuesto por parte del cliente.
- Modelo Relacional y Normalización (Modelo_Relacional_y_Normalizacion_TecnoService.docx): presenta el modelo relacional derivado del diagrama E-R y el proceso de normalización paso a paso hasta 3FN.
- Regla de negocio de control de stock: implementada como trigger (trg_validar_stock) en el script SQL, y replicada en acceso_datos.py para que el prototipo se comporte igual.
- Regla de negocio de aprobación de presupuesto: implementada como restricción CHECK (chk_presupuesto_aprobado_para_reparar) en el script SQL, y replicada en acceso_datos.py.
- ABM completo: se agregó la pestaña Clientes (alta, modificación y baja) y se completaron las operaciones de Modificar y Eliminar en Técnicos y Repuestos, que antes solo tenían alta.

7. Roles de Acceso a Nivel de Base de Datos

Conforme al apartado "Alcance: Qué No Se Solicita" de los lineamientos, el proyecto no implementa un sistema de login ni gestión de sesiones a nivel de aplicación. El control de acceso se resuelve exclusivamente a nivel del SGBD mediante tres roles de PostgreSQL creados en el script DDL:

- admin_role: control total sobre todas las tablas y secuencias.
- tecnico_role: lectura general y actualización de diagnóstico, estado y repuestos utilizados en las reparaciones.
- recepcion_role: alta de clientes y equipos, y creación de órdenes de trabajo.

Estos tres roles son coherentes con los tres tipos de usuario identificados en el relevamiento del sistema (Administrador, Técnico y Recepcionista).